Application des surnoms des enseignants - Step3

Dans cette étape, nous allons travailler dans une vue (le V du pattern MVC).

Qu'est qu'une vue ?

Une vue est un fichier qui contient du HTML combiné avec des balises dynamiques pour insérer des données générées par le serveur.
Ces fichiers sont généralement gérés par un moteur de templates, tel que `Edge` dans AdonisJS.

Qu'est ce qu'un gestionnaire de template ?

Un gestionnaire de templates (ou moteur de templates) est un outil ou une bibliothèque qui permet de générer du contenu HTML dynamique en combinant des fichiers de templates statiques avec des données dynamiques fournies par une application.

Ces gestionnaires simplifient le processus de création et de gestion des vues dans une application en séparant clairement la logique de présentation (HTML/CSS) de la logique métier.

Présentation des fichiers

Pendant cette étape, nous allons créer/modifier les fichiers suivants :

```
resources/
  css/
    app.css          ← modifié
  views/
    components/
      layout.edge    ← modifié
    pages/
      home.edge      ← modifié
    partials/
      header.edge    ← modifié
      footer.edge    ← créé (exercice)
public/
  img/               ← créé
```

Version statique de l'application des surnoms des enseignants

Pour commencer cette aventure avec le framework AdonisJS, nous allons récupérer la Version statique HTML5 / CSS3 de l'application des surnoms des enseignants.

Comme son nom l'indique, cette version statique ne contient que des fichiers HTML, CSS et 1 fichier JS (mais exécuté côté client).

Au cours de la construction de notre application, nous allons récupérer du HTML de cette version statique.

Cela nous permettra de nous concentrer sur l'apprentissage d'AdonisJS et pas sur le HTML5.

Images

Dans la version statique de l'application, il y a quelques icônes présents dans le répertoire `img/`.

Dans notre projet AdonisJS, nous allons créer un répertoire `public/` à la racine du projet et copier le répertoire `img/` à l'intérieur.

Fichier css

Nous allons copier le CSS ci-dessous pour le coller dans le fichier `app.css` présent dans le répertoire `resources/css`.

```
body {
  font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif;
  margin: auto;
  width: 80%;
}

nav {
  border: black solid 3px;
  background-color: hsl(0, 0%, 90%);
  height: 50px;
  display: flex;
  align-items: center;
  justify-content: center;
}

.container {
  height: 50%;
}

footer {
  border-top: black solid 1px;
  text-align: center;
  margin-top: 3vw;
}

table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
}

th,
td {
  padding: 1vw;
  border-bottom: #ddd solid 1px;
}

.containerOptions a,
```

```css
.user-head a {
  text-decoration: none;
  height: 100%;
  margin-left: 1vw;
}

.user-head {
  width: 100%;
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
}

.user-footer {
  text-align: right;
}

.left,
.rigth {
  display: inline-block;
}

.left {
  width: 70%;
}

label {
  display: inline-block;
  width: 10%;
}

.login label {
  display: inline-block;
  width: 12%;
}

.login-line {
  margin-bottom: 5px;
}

.container-header {
  display: flex;
  justify-content: center;
  align-items: center;
  padding-top: 5%;
}

.titre-header {
  width: 60%;
  height: 100%;
}

.login-container {
```

```
  width: 40%;
  height: 100%;
  display: flex;
  justify-content: end;
  align-items: center;
}

.login-container form {
  display: flex;
  justify-content: space-between;
}

.login-container label {
  display: none;
}

.login-container input,
.login-container .btn-login {
  width: 30%;
  display: inline-block;
  border: 1px solid #ccc;
  border-radius: 8px;
}

.login-container h3 {
  display: inline-block;
  width: 30%;
  text-align: right;
  margin-right: 1vw;
}

.login-container input {
  height: 4em;
  margin-bottom: auto;
  margin-top: auto;
}

.btn-login {
  background-color: #00b4cc;
}

.btn {
  text-align: center;
  color: white;
  width: 100%;
  border: none;
  cursor: pointer;
  height: 4.5em;
  border-radius: 8px;
}

.btn:hover {
  opacity: 0.8;
}
```

```css
.btn-logout {
  background-color: coral;
  padding: 20px;
  margin: 10px;
}

.text-danger {
  color: red;
}

.alert {
  padding: 15px;
  border-radius: 4px;
  font-size: 16px;
  margin: 10px 0;
}

.alert-success {
  background-color: #d4edda;
  border: 1px solid #c3e6cb;
  color: #155724;
}

.alert-destructive {
  background-color: #f8d7da;
  border: 1px solid #f5c6cb;
  color: #721c24;
  padding: 15px;
  border-radius: 4px;
  font-size: 16px;
  margin: 10px 0;
}

.field {
  margin-top: 10px;
  margin-bottom: 10px;
}

.radio {
  margin-top: 5px;
  margin-bottom: 15px;
}

.containerOptions a,
.containerOptions form,
.user-head a {
  text-decoration: none;
  height: 100%;
  margin-left: 1vw;
}
```

Modifier la vue 'home'

Nous allons modifier la vue `views/pages/home.edge` pour copier/coller une partie du html présent dans le fichier `index.html` de la version statique.

Voilà à quoi doit ressembler votre fichier `views/pages/home.edge`.

Ce code sera modifié dans les étapes suivantes au fur et à mesure des notions expliquées ensemble.

```
@layout()
<div class="container">
  <h3>Liste des enseignants</h3>
  <form action="#" method="post">
    <table>
      <thead>
        <tr>
          <th>Nom</th>
          <th>Surnom</th>
          <th>Options</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>Girardet Alain</td>
          <td>AGT</td>
          <td class="containerOptions">
            <a href="#">
              <img height="20em" src="./img/edit.png" alt="edit" />
            </a>
            <a href="javascript:confirmDelete()">
              <img height="20em" src="./img/delete.png" alt="delete" />
            </a>
            <a href="detailTeacher.html">
              <img height="20em" src="./img/detail.png" alt="detail" />
            </a>
          </td>
        </tr>
        <tr>
          <td>Mveng Antoine</td>
          <td>AMG</td>
          <td class="containerOptions">
            <a href="#">
              <img height="20em" src="./img/edit.png" alt="edit" />
            </a>
            <a href="javascript:confirmDelete()">
              <img height="20em" src="./img/delete.png" alt="delete" />
            </a>
            <a href="#">
              <img height="20em" src="./img/detail.png" alt="detail" />
            </a>
          </td>
        </tr>
        <tr>
```

```
            <td>Charmier Grégory</td>
            <td>GregLeBarbar</td>
            <td class="containerOptions">
              <a href="#">
                <img height="20em" src="./img/edit.png" alt="edit" />
              </a>
              <a href="javascript:confirmDelete()">
                <img height="20em" src="./img/delete.png" alt="delete" />
              </a>
              <a href="#">
                <img height="20em" src="./img/detail.png" alt="detail" />
              </a>
            </td>
          </tr>
        </tbody>
      </table>
    </form>
    <script src="js/script.js"></script>
  </div>
  @end
```

Il ne vous a pas échappé que la 1ère ligne commence par `@layout()`.
Nous allons commencer par comprendre ce qu'est un layout.

Qu'est ce qu'un layout

Un layout (ou mise en page en français) est une structure commune utilisée dans les applications web pour définir un modèle de base partagé par plusieurs pages. Il sert à encapsuler les éléments répétés, comme les en-têtes, pieds de page, barres de navigation, et autres composants communs.

L'utilisation des layouts permet de réduire la duplication de code et d'assurer une cohérence visuelle et structurelle sur l'ensemble des pages de l'application.

En version 7, AdonisJS propose nativement la directive `@layout()` pour utiliser un layout. C'est la méthode recommandée que nous allons utiliser.

Composant layout

Ce composant existe déjà et il est présent dans `ressources/views/components/layout.edge`.

Ce fichier est le **squelette HTML commun** à toutes les pages de l'application. Il définit la structure de base et les éléments répétés (en-tête, pied de page, scripts).

Vous pouvez modifier ce layout de la manière suivante :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Application des surnoms des enseignants</title>
    @vite(['resources/css/app.css', 'resources/js/app.js'])
    @stack('dumper')
  </head>

  <body>
    @include('partials/header')
    <main>
      @include('partials/flash_alerts')
      {{{ await $slots.main() }}}
    </main>
  </body>
</html>
```

Nous avons simplement changer la langue `lang="fr"` et le titre `<title>Application des surnoms des enseignants</title>` !

Ok mais il est important d'expliquer quelques points

La directive `@vite`

```
@vite(['resources/css/app.css', 'resources/js/app.js'])
```

Vite est l'outil qui gère les fichiers statiques (CSS, JS) de l'application.
Cette directive fait deux choses selon le contexte :

- **En développement** : elle connecte le navigateur au serveur Vite pour recharger automatiquement la page à chaque modification d'un fichier CSS ou JS.
- **En production** : elle pointe vers les fichiers compilés et optimisés (avec hash dans le nom pour le cache navigateur).

C'est l'équivalent moderne d'un simple `<link rel="stylesheet">` et `<script src="...">`, mais géré automatiquement selon l'environnement.

La directive `@stack`

```
@stack('dumper')
```

`@stack` définit un **emplacement réservé** dans le layout où du contenu peut être injecté depuis n'importe quelle vue.

Ici, le stack nommé `dumper` est réservé au système de débogage d'AdonisJS. Quand on utilise `@dump(maVariable)` dans une vue, AdonisJS injecte automatiquement ses scripts et styles dans ce stack.

> ⚠ Sans cette ligne dans le layout, `@dump` afficherait le contenu mais sans mise en forme ni interactivité (expand/collapse).

On reviendra sur `@dump` dans l'étape dédiée au débogage.

Les partials et la directive `@include`

```
@include('partials/header')
@include('partials/flash_alerts')
```

Un **partial** est un fragment de template réutilisable qui représente une partie d'une page.
Il permet de découper les vues en petits morceaux indépendants et maintenables.

`@include` insère simplement le contenu du fichier pointé à l'endroit de la directive — c'est un copier/coller au moment du rendu côté serveur.

Dans notre cas :

Partial	Fichier	Rôle
`header`	`resources/views/partials/header.edge`	Navigation et en-tête de page
`flash_alerts`	`resources/views/partials/flash_alerts.edge`	Messages flash (succès, erreurs)

Le partial `flash_alerts.edge` est fourni par le kit de démarrage v7. Il affichera les messages de retour à l'utilisateur (succès, erreurs). Nous l'utiliserons plus tard notamment pour la suppression.

Les slots : `{{{ await $slots.main() }}}`

C'est la ligne la plus importante du layout. Elle définit **l'endroit où le contenu spécifique à chaque page sera inséré**.

Comment ça fonctionne ?

Dans une vue qui utilise ce layout, le contenu est défini dans un bloc :

```
@layout()
  <h1>Contenu de ma page</h1>
  <p>Ce contenu sera injecté à la place du slot.</p>
@end
```

`$slots.main()` récupère ce bloc et l'injecte dans le layout à l'endroit de la directive.

Pourquoi `await` ?

Parce que le rendu du slot est une opération **asynchrone** — le contenu peut lui-même contenir des appels asynchrones (accès base de données, etc.).

Pourquoi trois accolades `{{{ }}}` ?

C'est la distinction fondamentale du moteur de templates Edge :

Syntaxe	Comportement	Quand l'utiliser
`{{ }}`	**Échappe** le HTML	Par défaut, pour afficher du texte
`{{{ }}}`	**Brut**, sans échappement	Pour injecter du vrai HTML

Exemple concret

```
{{-- Si la variable contient : <strong>Bonjour</strong> --}}
{{ maVariable }}
{{-- Le navigateur affiche littéralement : <strong>Bonjour</strong> --}}
{{-- (les chevrons sont convertis en entités HTML : &lt; et &gt;) --}}
{{{ maVariable }}}
{{-- Le navigateur affiche : Bonjour (en gras) --}}
{{-- (le HTML est interprété par le navigateur) --}}
```

Pourquoi cette distinction ?

L'échappement avec `{{ }}` est le comportement **par défaut et recommandé** car il protège contre les failles **XSS** (Cross-Site Scripting), c'est-à-dire l'injection de code malveillant par un utilisateur.

On n'utilise `{{{ }}}` que lorsqu'on a la **certitude absolue** que la source de la donnée est fiable.
C'est le cas ici : c'est AdonisJS lui-même qui génère le contenu du slot, pas un utilisateur.

> 🔐 **Règle à retenir** : `{{ }}` par défaut, `{{{ }}}` uniquement pour du HTML de confiance.

Création de deux partials pour le header et le footer

Partial header

Nous devons modifier le fichier `views/partials/header.edge` avec le code HTML suivant (qui est repris de l'application statique).

```
<header>
  <div class="container-header">
    <div class="titre-header">
      <h1>Surnom des enseignants</h1>
    </div>
    <div class="login-container">
      <form action="#" method="post">
        <label for="user"> </label>
        <input type="text" name="user" id="user" placeholder="Login" />
        <label for="password"> </label>
        <input type="password" name="password" id="password" placeholder="Mot de
passe" />
        <button type="submit" class="btn btn-login">Se connecter</button>
      </form>
    </div>
  </div>
  <nav>
    <h2>Zone pour le menu</h2>
      <a href="index.html">Accueil</a> 
    <a href="addTeacher.html">Ajouter un enseignant</a>
  </nav>
</header>
```

Partial footer

Exercice : A vous de créer un partial `footer.edge` avec le contenu suivant :

```
<footer>
  <p>Copyright GCR - bulle web-db - 2026-27</p>
</footer>
```

Note : Ne pas oublier de l'ajouter à votre layout !

Etat de l'application à la fin de cette étape

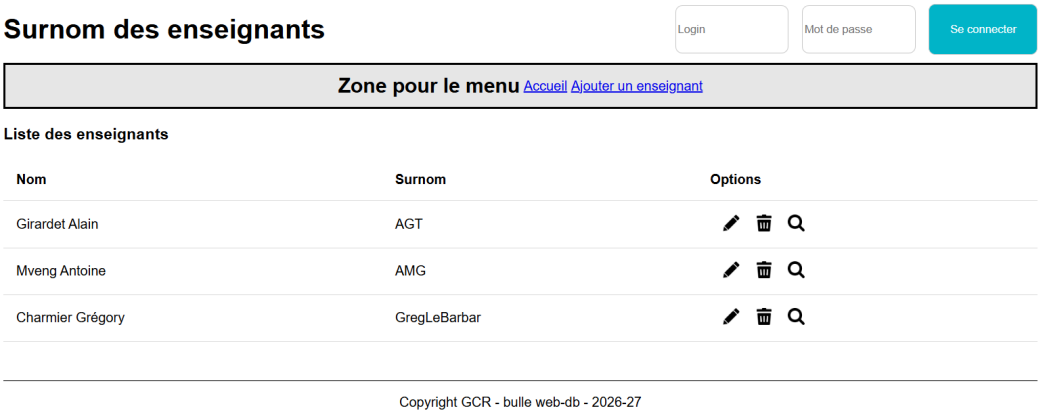
Pour voir le résultat des modifications effectuées, nous devons commencer par lancer le serveur de développement.

```
npm run dev
```

Voir la page http://localhost:3333/.

Notre application n'a qu'une seule page à savoir la homepage.

Et pour l'instant rien ne fonctionne !



Prochaine étape

Dans la prochaine étape step4, nous allons découvrir le contrôleur, puis rendre la homepage dynamique en utilisant davantage le gestionnaire de template edge.